

Module-1

1. **Explain the OSI reference model** in detail. Describe the functions of each layer with a neat diagram.

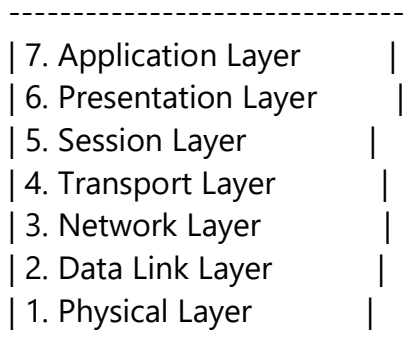
OSI Reference Model

Definition:

The **OSI (Open Systems Interconnection)** reference model is a **conceptual framework** developed by **ISO** for standardizing communication between different systems without requiring changes to hardware or software.

It divides the communication process into **seven layers**, each performing specific functions.

Neat Diagram:



Layer	Functions	Examples
7. Application Layer	Provides user interfaces and network services like file transfer, email, and web access.	HTTP, FTP, SMTP, DNS
6. Presentation Layer	Translates data between application and network format, performs encryption and compression.	JPEG, MPEG, SSL
5. Session Layer	Establishes, manages, and terminates sessions between applications.	NetBIOS, RPC
4. Transport Layer	Provides reliable end-to-end delivery, error control, and flow control.	TCP, UDP
3. Network Layer	Determines the best physical path for data; handles logical addressing and routing.	IP, Routers
2. Data Link Layer	Provides node-to-node delivery, framing, error detection, and flow control.	Switch, Ethernet, MAC
1. Physical Layer	Transmits raw bits over a physical medium; defines cables, voltages, and rates.	Hubs, Repeaters, Cables

2. **Compare the OSI model and the TCP/IP protocol suite.** Highlight their similarities and differences.

Feature	OSI Model	TCP/IP Model
Full Form	Open Systems Interconnection Model	Transmission Control Protocol / Internet Protocol Model

Feature	OSI Model	TCP/IP Model
Developed by	ISO (International Standards Organization)	DARPA (Defense Advanced Research Projects Agency), USA
Number of Layers	7 Layers	4 Layers
Layers	Application, Presentation, Session, Transport, Network, Data Link, Physical	Application, Transport, Internet, Network Interface
Purpose	Reference model for network communication	Practical model for data communication over the Internet
Protocol Dependency	Protocol-independent	Protocol-dependent (uses TCP, IP, HTTP, etc.)
Presentation and Session Layers	Present separately	Not present; their functions are part of the Application layer
Transport Layer	Supports both connection-oriented and connectionless services	Supports both TCP (connection-oriented) and UDP (connectionless)
Use	Used as a guideline to understand network concepts	Used for actual communication in real networks
Example Protocols	Not defined	HTTP, FTP, TCP, IP, UDP, SMTP, etc.

Diagram

OSI Model (7 Layers):

Application
Presentation
Session
Transport
Network
Data Link
Physical

TCP/IP Model (4 Layers):

Application
Transport
Internet
Network Interface

3. Define data communication. Explain the key components and characteristics of an effective data communication system.

Definition:

Data Communication is the process of **transmitting and receiving data** between two or more digital devices (such as computers, smartphones, or servers) through some form of **transmission medium** like cables or wireless signals.

It enables the exchange of information in the form of binary data (0s and 1s).

Key Components of Data Communication System:

1. **Message:**
The actual information or data to be communicated (e.g., text, image, audio, video).
2. **Sender (Transmitter):**
The device that sends the data message (e.g., computer, mobile phone).
3. **Receiver:**
The device that receives the message (e.g., another computer or printer).
4. **Transmission Medium:**
The physical path through which data travels from sender to receiver.
Examples: Twisted pair cable, fiber optic cable, radio waves, etc.
5. **Protocol:**
A set of **rules** that govern data communication, ensuring proper transmission and reception.
Examples: TCP/IP, HTTP, FTP, etc.

Characteristics of an Effective Data Communication System:

1. **Delivery:**
The system must deliver data to the correct destination and intended device.
2. **Accuracy:**
The data must be delivered **accurately**, without any alteration or loss during transmission.
3. **Timeliness:**
The data should be delivered **on time**, especially in real-time communication (like video calls).
4. **Jitter:**
It refers to the **variation in packet arrival time**. A good communication system should minimize jitter.
5. **Security (optional point for full marks):**
The data must be protected from unauthorized access or modification during transmission.

4.What is a computer network? Explain the different types of networks (LAN, MAN, WAN) with suitable examples.

Definition:

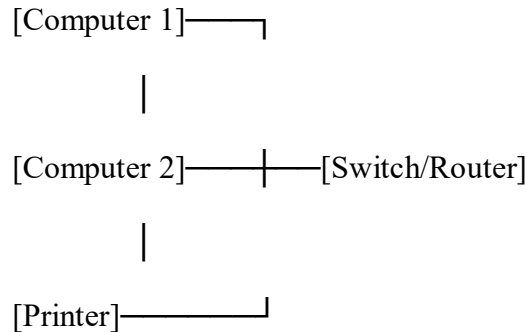
A **computer network** is a system in which **two or more computers or devices are connected** together to **share resources**, such as data, files, hardware (printers, scanners), and internet services.

These connections can be made using **wired** or **wireless** communication links.

Types of Computer Networks:

1. **LAN (Local Area Network):**
 - **Definition:**
A LAN is a network that connects computers and devices **within a small geographical area**, such as a single building, office, or campus.
 - **Features:**
 - High data transfer speed
 - Low cost of installation
 - Owned and maintained by a single organization
 - **Example:**
Computer network in a school, college lab, or an office building.

Diagram (optional for extra marks):



2. MAN (Metropolitan Area Network):

- **Definition:**
A MAN is a network that covers a **larger area than LAN** but **smaller than WAN**, usually across a **city or a large campus**.
- **Features:**
 - Medium data speed
 - Connects multiple LANs within a city
 - Often used by government or large organizations
- **Example:**
Networking between branches of a bank or between buildings of a university within the same city.

3. WAN (Wide Area Network):

- **Definition:**
A WAN covers a **large geographical area**, such as a **country or continent**, and connects multiple LANs and MANs.
- **Features:**
 - Long-distance communication
 - Lower data transfer rate compared to LAN and MAN
 - Uses public transmission lines like telephone systems or satellite links
- **Example:**
The **Internet** is the best example of a WAN.
Other examples: A multinational company's offices connected across countries.

Summary Table:

Type	Full Form	Coverage Area	Data Speed	Example
LAN	Local Area Network	Small area (building, office)	High	College lab network
MAN	Metropolitan Area Network	City or campus	Medium	City-wide bank network
WAN	Wide Area Network	Country or worldwide	Low	Internet

5. Describe the principles of network applications. Discuss network application architectures such as Client–Server and Peer-to-Peer models.

Principles of Network Applications:

Network applications are software programs that **run on different systems connected through a network** and enable communication and data exchange between them.

The design of such applications is based on the following principles:

1. **Process Communication:**
Applications consist of **processes** that communicate with each other using the network.
2. **Client and Server Roles:**
One process acts as a **client** (requests services), and the other acts as a **server** (provides services).
3. **Sockets Interface:**
Communication between client and server takes place using a **socket**, which acts as an endpoint for sending and receiving data.
4. **Protocols:**
Network applications use communication protocols such as **HTTP, FTP, SMTP, or DNS** to define how data is exchanged.
5. **Data Exchange:**
The application must ensure reliable and efficient data transfer between devices, often using **TCP or UDP** at the transport layer.

1. Client–Server Model

- **Definition:**
In the Client–Server model, one computer acts as a **server** that provides services or resources, and the other computers act as **clients** that request those services.
- **Features:**
 - Server is centralized and provides data or services.
 - Clients send requests and receive responses from the server.
 - Server can serve multiple clients simultaneously.
- **Examples:**
Web applications, Email systems, Online banking, File servers.

Diagram:

[Client 1] \

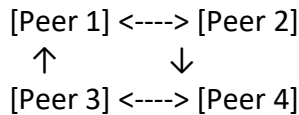
[Client 2] ----> [Server]

[Client 3] /

2. Peer-to-Peer (P2P) Model

- **Definition:**
In the Peer-to-Peer model, there is **no central server**. Each device (peer) acts as both a **client** and a **server**, sharing resources directly with other peers.
- **Features:**
 - Decentralized network structure.
 - Each peer can both request and provide resources.
 - Scalable and cost-effective.
- **Examples:**
BitTorrent, Blockchain, Skype, File sharing systems.

Diagram:



Difference between Client–Server and Peer-to-Peer Models:

Aspect	Client–Server	Peer-to-Peer (P2P)
Structure	Centralized	Decentralized
Control	Controlled by server	Shared among peers
Cost	Expensive due to dedicated server	Low cost
Reliability	Failure of server affects all clients	No single point of failure
Example	Web, Email	BitTorrent, Blockchain

6. Explain the concept of transport services available to network applications. How do these services differ in the Internet?

Concept of Transport Services:

The **transport layer** is responsible for providing **communication services directly to the application layer** in a computer network.

It ensures that data is delivered **reliably, accurately, and in the correct sequence** between two communicating processes (applications).

Transport services provide logical communication between application processes running on different hosts.

Transport Services Available to Network Applications:

The main transport services offered by the Internet are provided through two protocols:

1. Transmission Control Protocol (TCP)

Type: Connection-oriented service

Features:

- **Reliable data transfer:** Ensures data is delivered without loss, error, or duplication.
- **Connection establishment:** A connection is established between sender and receiver before data transfer (3-way handshake).
- **Flow control:** Prevents sender from overwhelming the receiver.
- **Congestion control:** Avoids network overload.
- **Data sequencing:** Ensures packets arrive in correct order.
- **Error detection and correction:** Detects and retransmits lost or corrupted packets.

Examples of applications using TCP:

HTTP (Web), FTP (File transfer), SMTP (Email), Telnet.

2. User Datagram Protocol (UDP)

Type: Connectionless service

Features:

- **No connection setup:** Data is sent directly without prior connection.
- **Unreliable delivery:** Packets may be lost, duplicated, or arrive out of order.
- **No flow or congestion control:** Simple and faster communication.
- **Low overhead:** Suitable for real-time and streaming applications.

Examples of applications using UDP:

DNS, Voice over IP (VoIP), Video streaming, Online gaming.

7. Discuss the working of HTTP protocol in detail. Differentiate between persistent and non-persistent HTTP connections using diagrams.

HTTP Protocol:

HTTP (HyperText Transfer Protocol) is an **application layer protocol** used for communication between a **client (web browser)** and a **server (web server)** over the World Wide Web (WWW).

It defines how messages are **formatted and transmitted**, and how web servers and browsers should respond to various commands.

Working of HTTP:

1. **Client–Server Model:**

HTTP works on a request–response model. The **client** sends an HTTP request to the **server**, and the server sends an HTTP response back to the client.

2. **Transport Layer:**

HTTP uses **TCP** as the transport protocol, usually on **port 80** (or 443 for HTTPS).

3. **HTTP Request Message:**

GET /index.html HTTP/1.1

Host: www.example.com

4. **HTTP Response Message:**

HTTP/1.1 200 OK

Content-Type: text/html

<html>...</html>

5. **Common HTTP Methods:**

- **GET:** Request data from the server.
- **POST:** Send data to the server.
- **PUT:** Update data on the server.
- **DELETE:** Remove data.

Difference Between Persistent and Non-Persistent HTTP

Aspect	Non-Persistent HTTP	Persistent HTTP
TCP Connection	New connection for every request	One connection for multiple requests
Performance	Slower	Faster
Overhead	High	Low
Connection Closing	Closed after each response	Remains open for several responses
Used In	HTTP/1.0	HTTP/1.1 and above

8. Describe the HTTP request and response message formats with examples.

HTTP Messages:

HTTP communication between client and server takes place using **two types of messages**:

1. **HTTP Request Message** (sent by client)
2. **HTTP Response Message** (sent by server)

1. HTTP Request Message

Format:

<Method> <URL> <HTTP Version>

<Header Fields>

<Blank Line>

<Optional Message Body>

Example:

GET /index.html HTTP/1.1

Host: www.example.com

User-Agent: Mozilla/5.0

Accept: text/html

Explanation:

- **GET** – Method (asks data from the server)
- **/index.html** – Requested resource
- **HTTP/1.1** – Version
- **Header fields** – Give additional info (Host, User-Agent, etc.)

2. HTTP Response Message

Format:

<HTTP Version> <Status Code> <Reason Phrase>

<Header Fields>

<Blank Line>

<Optional Message Body>

Example:

HTTP/1.1 200 OK

Date: Fri, 07 Nov 2025 12:00:00 GMT

Server: Apache/2.4.1

Content-Type: text/html

Content-Length: 1024

<html>

<body>Welcome</body>

</html>

Explanation:

- **HTTP/1.1** – Version
- **200 OK** – Status code and reason phrase
- **Header fields** – Extra info (date, server, content type)
- **Message body** – Actual data (webpage content)

9. What are cookies? Explain how cookies enable user–server interaction and session management on the web.

Definition:

A **cookie** is a small piece of data stored on the **client's browser** by a **web server** to remember information about the user.

Cookies help maintain **state information** across multiple HTTP requests, as **HTTP is a stateless protocol**.

Purpose of Cookies:

- To **identify users**
- To **store user preferences**
- To **manage sessions** (like login, shopping cart, etc.)

How Cookies Work (User-Server Interaction):

1. **Client request:**

The user sends an HTTP request to the web server (e.g., login page).

2. **Server response with cookie:**

The server sends an HTTP response containing a `Set-Cookie` header.

Set-Cookie: userID=12345; Expires=Wed, 10 Nov 2025 12:00:00 GMT

3. **Browser stores the cookie:**

The browser saves this cookie in local storage.

4. **Subsequent requests:**

For every new request to the same server, the browser automatically includes the cookie:

Cookie: userID=12345

5. **Server identifies the user:**

The server uses the cookie information to recognize the user and maintain the session.

Cookie Example:

Request:

GET /home HTTP/1.1

Host: www.example.com

Cookie: userID=12345

Response:

HTTP/1.1 200 OK

Set-Cookie: theme=dark; Expires=Fri, 15 Nov 2025 12:00:00 GMT

10. What is Web Caching? Explain how caching improves web performance and reduces network load with a suitable example.

Definition:

Web caching is the process of **storing copies of web resources (like web pages, images, or files)** temporarily in a **cache** (browser, proxy, or server) so that future requests for the same data can be served faster.

Working of Web Caching:

1. When a user requests a webpage for the first time, the **browser or proxy** sends the request to the **web server**.
2. The **server responds** with the web page and it is **stored in the cache**.
3. When the **same webpage** is requested again, it is **fetches directly from the cache** instead of the original server.

Example:

- When a user visits `www.example.com/logo.png` for the first time, it is downloaded from the server and stored in the browser cache.
- When the same image is requested again, it is loaded from the **cache**, not the server — making it faster.

Advantages:

- Reduces **response time** (faster page loading).
- Decreases **network traffic**.
- Lowers **server load**.
- Improves **overall web performance**.

11. Distinguish between connectionless and connection-oriented multiplexing and demultiplexing in the transport layer.

Definition:

Multiplexing is the process of **collecting data** from multiple applications and sending them over a single network channel.

Demultiplexing is the process of **delivering received data** from the network to the correct application process.

1. Connectionless Multiplexing and Demultiplexing

- Used by **UDP (User Datagram Protocol)**.
- No connection is established before data transfer.
- Each segment is treated as an **independent packet**.
- The transport layer identifies the destination process using **port numbers** (source and destination).
- No guarantee of delivery, order, or reliability.

Example:

Online video streaming, DNS query (uses UDP).

2. Connection-Oriented Multiplexing and Demultiplexing

- Used by **TCP (Transmission Control Protocol)**.
- A **connection** is established between sender and receiver before data transfer.
- Each segment belongs to a particular connection identified by a **4-tuple**: (Source IP, Source Port, Destination IP, Destination Port).
- Ensures **reliable, ordered, and error-free** data delivery.

Example:

Email (SMTP), File transfer (FTP), Web browsing (HTTP).

Difference Between Connectionless and Connection-Oriented Multiplexing

Aspect	Connectionless (UDP)	Connection-Oriented (TCP)
Protocol Used	UDP	TCP
Connection Setup	No connection established	Connection established before transfer
Reliability	Unreliable	Reliable

Aspect	Connectionless (UDP)	Connection-Oriented (TCP)
Data Order	May arrive out of order	Delivered in correct order
Identification	Using (Dest. IP, Dest. Port)	Using (Src IP, Src Port, Dest IP, Dest Port)
Overhead	Low	High
Example	DNS, Video Streaming	HTTP, FTP, Email

12. Explain different types of network application architectures and contrast their features.

Definition:

A **network application architecture** defines how **network processes are organized and interact** to provide communication between devices or users.

There are **two main types** of network application architectures:

1. Client–Server Architecture

- In this model, there is a clear distinction between **clients** and **servers**.
- The **server** provides services or resources, and the **client** requests them.

Working:

1. The **client** sends a request to the **server**.
2. The **server** processes the request and sends the response.

Examples:

- Web services (HTTP)
- Email (SMTP, IMAP)
- File transfer (FTP)

Features:

- Centralized control.
- Scalable (multiple clients can connect).
- Server can become overloaded.
- Secure and easy to manage.

2. Peer-to-Peer (P2P) Architecture

- There is **no fixed client or server**.
- Each host (peer) acts as both a **client and a server**.
- Peers communicate directly and share resources among themselves.

Examples:

- BitTorrent
- Skype
- Blockchain networks

Features:

- Decentralized control.
- Highly scalable.
- Difficult to manage and less secure.
- Resources are shared directly among users.

Difference Between Client–Server and Peer-to-Peer Architectures

Feature	Client–Server	Peer-to-Peer (P2P)
Structure	Centralized	Decentralized
Control	Server controls data	All peers share control
Scalability	Moderate	High
Reliability	High (if server is up)	Depends on active peers
Examples	Web, Email, FTP	BitTorrent, Skype
Security	Easier to manage	Harder to secure

